BASE DE DATOS II
PARCIAL #1

Modalidad: Trabajo en equipo (máximo 2 estudiantes)
Duración: 2 semanas
Entrega: Moodle

Estudiantes: Cristian Diaz, Roberto Lasso

Competencias a evaluar

El estudiante será capaz de:

- Diseñar modelos de datos complejos aplicando cardinalidad correcta.
- Construir diagramas entidad–relación.
- Aplicar normalización hasta 3FN.
- Implementar bases de datos usando Docker.
- Utilizar DDL, DML y CRUD.
- Utilizar VISTAS SQL.
- Justificar decisiones técnicas como DBA.

PARTE I – Diagrama Entidad Relación (10 pts)

Usted ha sido contratado como **Administrador de Base de Datos (DBA)** para diseñar la solución de datos de una plataforma empresarial.

Caso 1: Sistema Empresarial de Servicios Digitales

El sistema debe cubrir:

- Gestión de usuarios
- Autenticación
- Servicios contratados
- Pagos
- Auditoría de accesos

Actividades:

- Diseñar un diagrama entidad relación(**DER) completo** que incluya:

a) Entidades
b) Atributos
c) Relaciones

d) Cardinalidades (1:1, 1:N, N:M)
e) Entidades asociativas cuando aplique

- El modelo debe reflejar **reglas de negocio reales**

Implementamos una base de datos relacional que se encarga de la gestión de usuarios, autenticarlos, los servicios contratados, pagos y accesos. El script define las tablas, datos integrados y la creación de vistas de negocio, seguridad y auditoria

Las tablas principales son:

Rol: almacena los roles del sistema (id_rol PK, nombre_rol único, descripción, nivel_acceso, código_rol).

Usuario: datos de los usuarios (id_usuario PK, nombre, apellido, email único, password, estado_cuenta, fecha_registro, ultima_conexion, id_rol).

Sesion: registro de sesiones (id_sesion PK, id_usuario, fecha_inicio/fin, ip_acceso, dispositivo, tipo_sesion, token_sesion único).

Servicio Contratado: relación entre usuarios y servicios contratados (id_servicio_contratado PK, id_usuario FK, id_servicio FK, fecha_contratacion, fecha_fin_contrato, precio_acordado, estado_contrato, codigo_contrato único).

Pagos: pagos realizados (id_pago PK, id_usuario FK, id_servicio_contratado FK opcional, fecha_pago, monto con CHECK>=0, metodo_pago ENUM, estado_pago ENUM, transaccion_id único).

Auditoria Acceso: registros de auditoría (id_auditoria PK, id_usuario FK, fecha_acceso, accion, resultado, ip_origen, es_critico boolean, evento_id único).

Las relaciones entre entidades se da mediante una clave foranea entre otros elementos para asegurar integridad y consistencia. Algunos diseños y restricciones importantes son:

Uso de auto increment.

Campos con restricción.

ENUM para campos con valores acotados.

CHECK en columnas numericas.

El script incluye espacios para cada tabla. Estos datos permiten probar consultas, vistas y operaciones CRUD sin requerir datos externos.

Algunas operaciones CRUD son:

Create.

Update.

Read.

Delete.

Algunas vistas con propósitos específicos son:

vista_usuarios_servicios_pagos

vista_usuarios_segura

vista_auditoria_accesos_criticos

PARTE II – Implementación de Base de Datos con Docker (30 pts)

La empresa exige que toda la infraestructura de base de datos sea desplegada mediante Docker, garantizando portabilidad y estandarización.

Uso de Base de Datos con Docker (10 pts)

Requerimientos obligatorios

- Crear un contenedor Docker que levante un motor de base de datos relacional (MySQL o MariaDB).
- Utilizar los comandos de docker visto en clase de:
  - Ejecución de contenedor
  - Creación de contenedor
  - La asignación de nombre, variable, puerto e imagen
  - Estado actual del contenedor
  - Visualizar los eventos de logs
  - Conectar al contenedor por CLI
- En la creación del contenedor definir:
  - Nombre del contenedor
  - Usuario
  - Contraseña
  - Puerto expuesto

Entregables

- Comando Docker documentado
- Evidencia de que la base de datos está operativa

<u>NOTA:</u> No se aceptan instalaciones locales fuera de Docker

Commando de Docker:

docker run -d --name parcial1 -e MYSQL_ROOT_PASSWORD=rootad123! -e MYSQL_DATABASE=parcial -e MYSQL_USER=Ender -e MYSQL_PASSWORD=ballers134! -p 3307:3306 mariadb:latest

Resultado:

Unable to find image 'mariadb:latest' locally

latest: Pulling from library/mariadb

01d7766a2e4a: Pull complete

899bae0d402d: Pull complete

633047087aaf: Pull complete

15bc38add3f6: Pull complete

ab1b7f466f6e: Pull complete

a4dd30b1fad7: Pull complete

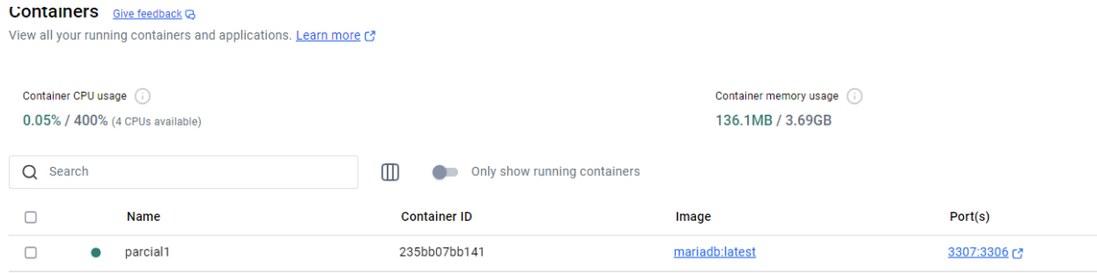
35c935f50ac0: Pull complete

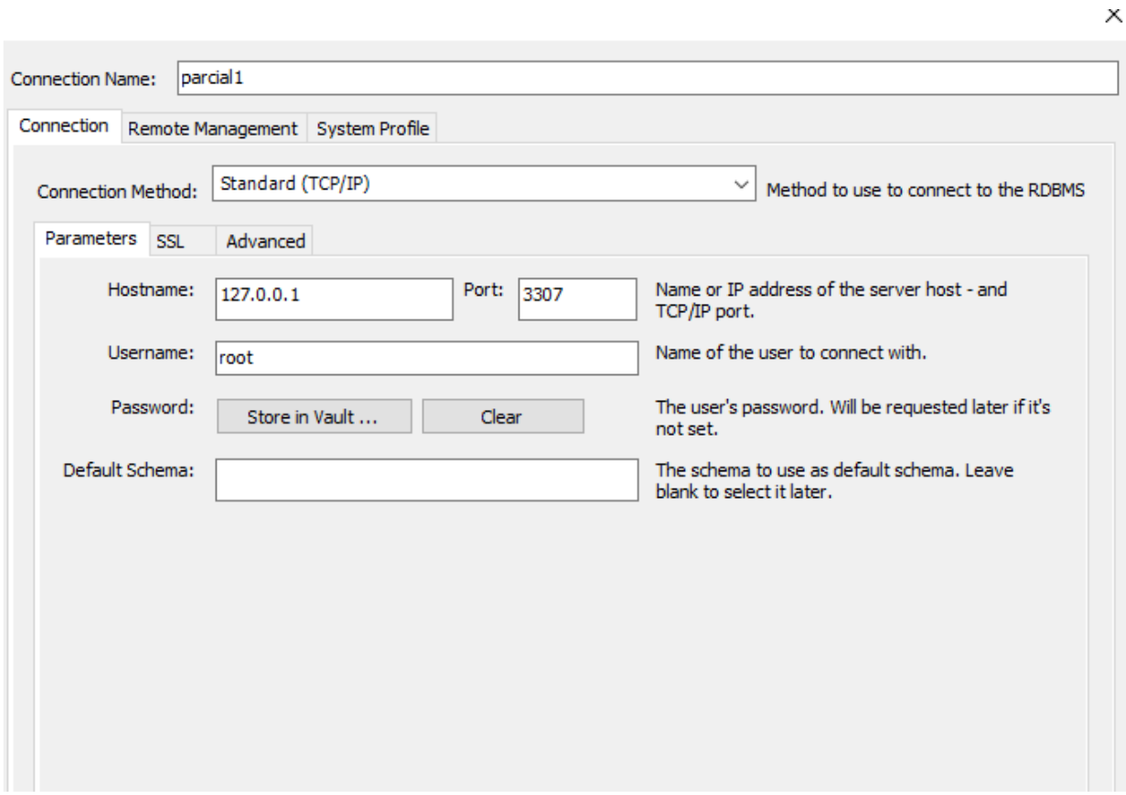
2d6698bfc424: Pull complete

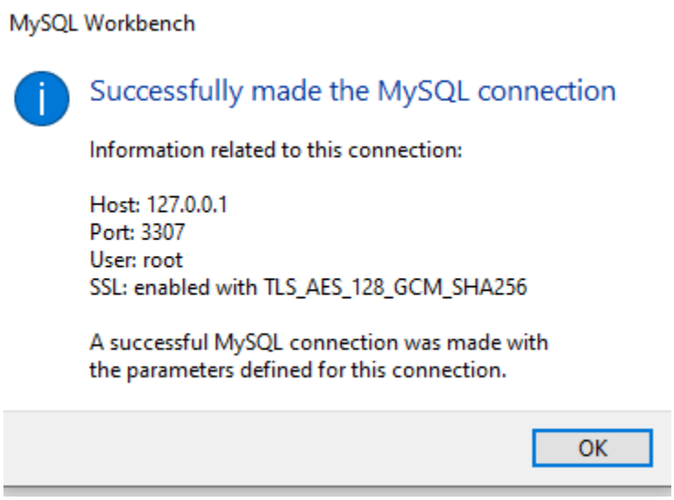
Digest: sha256:b1cb255a9939d28a1856815f0de6046c20c28c21b92a9f2696bc782b247a47ee

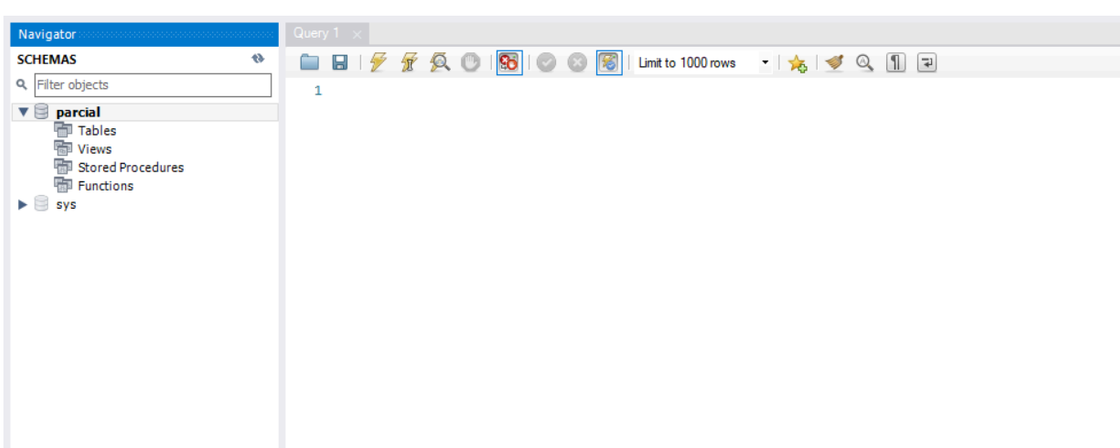
Status: Downloaded newer image for mariadb:latest

235bb07bb141da8bdab7db14c428fb8da890e1d1d0b93472996b4633bc6b80a1


Containers
Give feedback
View all your running containers and applications. Learn more
Container CPU usage
0.05% / 400% (4 CPUs available)
Container memory usage
136.1MB / 3.69GB
Search
Only show running containers
Name
Container ID
Image
Port(s)
parcial1
235bb07bb141
mariadb:latest
3307:3306

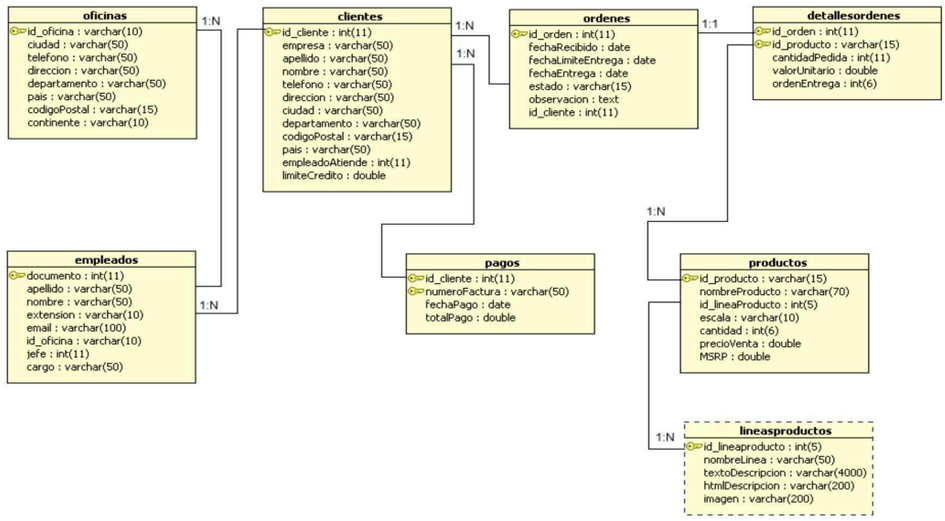

Connection Name: parcial1
Connection
Remote Management
System Profile
Connection Method: Standard (TCP/IP)
Method to use to connect to the RDBMS
Parameters
SSL
Advanced
Hostname: 127.0.0.1
Port: 3307
Name or IP address of the server host - and TCP/IP port.
Username: root
Name of the user to connect with.
Password:
Store in Vault ...
Clear
The user's password. Will be requested later if it's not set.
Default Schema:
The schema to use as default schema. Leave blank to select it later.


MySQL Workbench
Successfully made the MySQL connection
Information related to this connection:
Host: 127.0.0.1
Port: 3307
User: root
SSL: enabled with TLS_AES_128_GCM_SHA256
A successful MySQL connection was made with the parameters defined for this connection.
OK


Navigator
SCHEMAS
Filter objects
parcial
Tables
Views
Stored Procedures
Functions
sys
Query 1
Limit to 1000 rows
1

Caso 2: Sistema de punto de ventas (20 pts)

Basándonos en el siguiente diagrama entidad relación, realice las siguientes bases de datos.



El diseño base de datos debe tener como mínimo:

- Creación de las estructuras de tablas y relaciones.
- Se deberá agregar por cada tabla, 2 columnas adiciones, qué maneje los diferentes tipos de datos como: tipo lista, tipo si/no y columnas únicas.
- Se debe contar con un mínimo de 10 registros o más por cada tabla.
- Ejecutar con el comando SQL la estructura o descripción de cada tabla.

Primero se creó y diseñó las tablas principales:
Oficinas.
Empleados.
Clientes.
Lineasproductos.
Productos.
Ordenes.
Detallesordenes.
Pagos.

cada tabla se creó usando sentencias DDL como:
Primary Key
Foreign Key
Restricciones Not Null
Restricciones Unique
Campos tipo lista con ENUM
Campos booleanos
Columnas adicionales según se solicite.

Se cumplió el requerimiento de agregar por lo menos dos columnas adicionales por tabla como:
Campos ENUM:
tipo_oficina
estado_empleado
categoria_cliente
categoria_principal
estado_producto
tipo_envio
esstado_detalle
metodo_pago

también campos booleanos:
activa
es_gerente
activo
disponible_online

es_oferta
es_prioritaria
es_devolucion
pago_confirmado

Campos con restricción:
codigo_unico_oficina
numero_seguro_social
email_cliente
codigo_linea_unico

Los registros insertados por tabla fueron más de 10 por cada una por lo que cumplen con el mínimo exigido. se incluyeron distintos paises, empleados asociados a oficinas, clientes asignados a empleados, productos organizados por lineas de productos, ordenes vinculadas a clientes, detalles de ordenes relacionados con productos y pagos asociados a clientes.

La integridad del modelo funciona de la siguiente manera:
empleados>oficinas
clientes>empleados
productos>lineasproductos
ordenes>clientes
detallesordenes>ordenes y productos
pagos>clientes

de esta manera el sistema se mantiene consistente en las relaciones comerciales.

Los puntos **(III y IV)** se basará bajo el contexto del **caso 1** **“Sistema Empresarial de Servicios Digitales”**

PARTE III - Normalización y diseño lógico (20 puntos)

Normalización de la Base de Datos (10 pts)

- Aplicar normalización hasta Tercera Forma Normal (3FN).
- Explicar con sus propias palabras:
    - Qué problemas se resolvieron en cada etapa
    - Por qué el diseño final es óptimo

NOTA: Respuestas copiadas de internet o sin justificación técnica no suman puntos.

Implementación SQL – DDL (10 pts)

- Crear todas las tablas del modelo usando SQL.
- Cada tabla debe incluir: ○ Clave primaria
    - Claves foráneas
    - Restricciones (NOT NULL, UNIQUE, CHECK) ●

Por cada tabla:
    - 1 campo tipo lista (ENUM o tabla catálogo)
    - 1 campo tipo booleano
    - 1 restricción UNIQUE

El diseño final resulta óptimo ya que se basa en:

Reducción de redundancia.

Mayor integridad de datos.

Facilidad de mantenimiento.

Escalabilidad.

Consistencia estructural.

PARTE IV – DML, CRUD y Vistas (40 puntos)

Inserción de Datos – DML (10 pts)

- Insertar **mínimo 10 registros por tabla**.
- Los datos deben ser:
    - Realistas
    - Coherentes
    - Relacionados correctamente

Operaciones CRUD (15 pts)

Seleccionar **una entidad principal** del sistema y realizar:

- CREATE → Inserción
- READ → Consulta filtrada
- UPDATE → Actualización controlada
  DELETE → Eliminación lógica o física (justificar)

Se deben entregar los **scripts SQL** y explicar el propósito de cada operación.

Creación y Uso de VISTAS SQL (15 pts)

Requerimientos obligatorios

Crear **mínimo 3 VISTAS**, cumpliendo:

1. **Vista de negocio**
    0 Consolidación de datos relevantes para toma de decisiones ○ Ejemplo: usuarios + servicios + pagos

2. **Vista de seguridad**
    0 Oculta datos sensibles (contraseñas, tokens, etc.)

3. **Vista de auditoría**
    0 Muestra accesos, fechas, estado de usuarios o transacciones

Evaluación

- Correcta construcción de la vista
- Uso real (SELECT sobre la vista)

- Justificación de su utilidad

Los datos insertados cumplen con las siguientes características:

- Son realistas (nombres, correos, servicios, montos y fechas coherentes).
- Son consistentes con la lógica del negocio.
- Están correctamente relacionados mediante claves foráneas.
- Respetan todas las restricciones definidas.

Se insertaron registros en las tablas:

- ROL
- USUARIO
- SERVICIO
- SERVICIO_CONTRATADO
- PAGO
- AUDITORIA_ACCESO
- SESION

Operaciones CRUD

Se seleccionó como entidad principal la tabla USUARIO, ya que es una de las entidades centrales del sistema y se relaciona con múltiples módulos (servicios, pagos, auditoría y sesiones).

CREATE

Se realizó la inserción de un nuevo usuario utilizando INSERT INTO, agregando un registro con todos los campos obligatorios y respetando las restricciones del modelo.

Propósito:

Permitir el registro de nuevos usuarios dentro del sistema empresarial.

READ

Se ejecutó una consulta SELECT con filtro específico, incluyendo condiciones y, en algunos casos, JOIN con la tabla ROL para obtener información completa del usuario.

Propósito:

Permitir la visualización controlada de usuarios activos o con un rol específico, útil para gestión administrativa y toma de decisiones.

UPDATE

Se realizó una actualización mediante UPDATE para modificar datos específicos de un usuario (por ejemplo, estado de cuenta o rol).

Propósito:

Permitir cambios administrativos sin afectar otras entidades relacionadas.

La actualización se realizó con condición WHERE para evitar modificaciones masivas accidentales.

DELETE

En lugar de realizar una eliminación física (DELETE), se implementó una eliminación lógica, modificando el estado del usuario (por ejemplo, a “inactivo” o “suspendido”).

Justificación técnica:

Evita pérdida de información histórica.

Mantiene integridad referencial con pagos, servicios y auditoría.

Permite reactivar el usuario en el futuro si es necesario.

Previene errores en reportes históricos.

Creación y usos de vistas SQL

1. Vista de Negocio

Esta vista consolida información relevante combinando usuarios, servicios contratados y pagos.

Propósito:

Facilitar la toma de decisiones.

Visualizar qué usuarios tienen servicios activos.

Analizar ingresos generados por usuario o por servicio.

Simplificar consultas complejas mediante JOIN.

Se realizaron consultas SELECT sobre la vista para demostrar su funcionamiento real.

2. Vista de Seguridad

Se creó una vista que oculta datos sensibles como:

Contraseñas

Tokens

Información privada

Propósito:

Permitir acceso limitado a aplicaciones o empleados sin exponer información crítica.

Aplicar el principio de mínimo privilegio.

Mejorar la seguridad del sistema.

3. Vista de Auditoría

Se creó una vista enfocada en registros de auditoría y accesos.

Incluye información como:

Usuario

Fecha de acceso

Tipo de acción

Resultado del evento

Indicador de evento crítico

Propósito:

Monitorear accesos al sistema.

Detectar intentos fallidos o acciones sospechosas.

Apoyar procesos de control interno y seguridad informática.

Lineamiento e información a entregar

1. Desarrollar cada uno de los puntos
2. Crear el repositorio de carpeta ***"PARCIAL_01"***
3. Subir la copia de toda la base de datos
4. Capturas de cada de los procesos (Sentencia SQL y resultado)
5. Documentar todo el proceso y agregar las lecciones aprendidas.

Se puede realizar individual o en equipo con **un máximo de 2 integrantes,** con la condición qué por cada integrante adicional se deben agregar 10 registros en cada módulo, en caso de no cumplir con esta cláusula, se le baja automáticamente **-20 puntos de la nota final.**

Se debe subir a la plataforma el link o todos los documentos.

¡Buena Suerte!